

Taxonomy of Work Capstone Project

Building Task and Skill Taxonomies from Public Job Postings

Introduction

In this capstone project, you will use Claude Code—a command-line AI coding assistant—together with free LLM APIs to build a taxonomy of work from public job postings. One team will develop a taxonomy of tasks (what work gets done), while the other will develop a taxonomy of skills (what capabilities the work requires). In the second half of the course, the two teams will integrate their outputs to create a system that, given a business initiative, can answer two key questions: **What tasks does this initiative involve, and what skills are needed to carry them out?**

You will follow a general framework: extract structured statements from raw text, represent them semantically, cluster them into topics, label the topics, and organize them into a hierarchy. However, the choice of methods at each stage is up to you. Your solution may be fully algorithmic, LLM-assisted, or agentic. Through this project, you will learn not only how to code, but also how to design an end-to-end pipeline, justify methodological choices, and rigorously evaluate the quality of the results.

Course Structure

Duration: 15 weeks (7 sprints × 2 weeks each, plus final week)

Teams: 2 sections working with distinct but parallel problems, converging in Sprint 5. Each section has two teams

Domain: Workforce analytics — you'll work with public job postings and reference taxonomies (O*NET, ESCO, Lightcast)

Tools: Claude Code (shared account with 20X usage limit) for coding; free-tier LLM APIs (e.g. Google AI Studio / Gemini, GitHub Models, Groq) where the pipeline uses an LLM; open-source embedding and clustering libraries

Team Assignments

Team	Taxonomy	Reference Standard
Section A (Two teams)	Task taxonomy — a hierarchy of work activities derived from the responsibilities described in postings	O*NET task statements and Generalized Work Activities

Team	Taxonomy	Reference Standard
Section B (Two teams)	Skill taxonomy — a hierarchy of capabilities derived from the requirements described in postings	ESCO skills, Lightcast Open Skills
Joint (Sprints 5–7)	Task–skill mapping and the initiative → tasks → skills demo	O*NET task–skill and occupation–skill links

Both sections work from the same corpus and follow the same framework; they differ in what they extract (tasks vs. skills). Agreeing early on a shared data contract and a common representation is what makes the Sprint 5 join possible.

Sprint Overview

Sprint	Weeks	Starting Problem*	Skills Learned
1	1-2	Data Acquisition & Exploration	Claude Code basics, public data APIs, understanding job posting data
2	3-4	Text Extraction	Turning free text into atomic statements (rules, NLP, or LLM), caching, quality measurement
3	5-6	Representation & Clustering	Text embeddings, clustering methods, stability analysis
4	7-8	Labeling & Hierarchy	Topic naming, coherence checks, hierarchy construction, evaluation vs. reference taxonomies
5	9-10	Task–Skill Mapping (joint)	Cross-taxonomy linking, co-occurrence, retrieval evaluation
6	11-12	Integration & Polish	End-to-end pipeline, initiative → skills demo, error handling
7	13-14	Final Demo Preparation	Packaging and presentation
—	15	Final Presentations	Demo Day

***Adaptive Sprint Tasks:** The problems listed above are starting points. Actual sprint tasks will be adjusted based on each team's proficiency demonstrated in the previous sprint. Teams showing strong progress may receive more challenging tasks; teams needing more practice may receive reinforcement tasks before moving on.

Sprint Details

The sprint tasks below are **sample starting points**. Instructors will assign actual tasks at the start of each sprint based on team progress.

Sprint 1: Data Acquisition & Exploration (Weeks 1-2)

Sample Goal: Assemble a shared job-posting corpus and agree on the data contract

Sample Tasks:

- Set up Claude Code environment and a shared repository
- Collect a corpus of job postings from approved public sources (e.g. Kaggle dumps, Adzuna API, USAJobs API); load into a shared database
- Load O*NET and ESCO reference tables alongside it
- Use Claude Code to generate summary statistics: postings by industry, seniority, length; duplicate rate; language mix
- Agree the shared schema (see Data Contract) with the other group

Deliverable: Populated database, data exploration notebook with documented findings, agreed data contract

Sprint 2: Text Extraction (Weeks 3-4)

Sample Goal: Turn each posting into atomic task statements (Section A) or skill statements (Section B)

Sample Tasks:

- Choose an extraction approach — section parsing, sentence splitting, NLP, LLM prompting, or a combination — and justify it
- Produce one atomic statement per task or skill mentioned in a posting
- If using an LLM, add caching and rate-limit handling so re-runs are free and quota-safe
- Hand-label 100 postings to measure extraction precision/recall

Deliverable: Statements table in the shared database, extraction quality report

Sprint 3: Representation & Clustering (Weeks 5-6)

Sample Goal: Section statements into leaf topics

Sample Tasks:

- Choose a representation (e.g. sentence embeddings) and store it in the shared database
- Choose and fit a clustering method (e.g. BERTopic, HDBSCAN, k-means, agglomerative); compare at least two
- Run a parameter sweep and report cluster stability
- Decide whether a statement may belong to more than one topic, and implement that assignment

Deliverable: Topic and topic-membership tables, stability report with chosen parameters

Sprint 4: Labeling & Hierarchy (Weeks 7-8)

Sample Goal: Name every leaf topic and build a multi-level hierarchy

Sample Tasks:

- Name and describe every leaf topic (keywords, LLM, or manual review); reject vague or mixed topics
- Build parent levels above the leaves — options include hierarchical clustering, similarity thresholds, LLM-guided merging, or an agentic approach
- Evaluate leaf topics against O*NET (Group A) or ESCO/Lightcast (Group B): coverage, purity, nearest-neighbor agreement
- Review the hierarchy by hand and document where the method over- or under-merged

Deliverable: Labeled, hierarchical taxonomy plus evaluation report

Sprint 5: Task–Skill Mapping (Weeks 9-10, joint)

Sample Goal: Link the two taxonomies

Sample Tasks:

- Design a task–skill affinity score — candidate signals include semantic similarity, co-occurrence within postings, and O*NET priors
- Build a small gold set of 30 initiatives with expected skills; measure precision/recall of the mapping
- Implement initiative → tasks → skills end to end; the decomposition step may be LLM-based, retrieval-based, or manual

Deliverable: Task–skill mapping table, gold set, evaluation notebook

Sprint 6: Integration & Polish (Weeks 11-12)

Sample Goal: One command runs the whole pipeline; one interface answers the question

Sample Tasks:

- Package extraction, clustering, labeling, and mapping as a reproducible pipeline with config files
- Build a simple UI or notebook: enter an initiative, get ranked tasks and skills with supporting postings
- Add error handling, logging, and (if using LLMs) token/cost accounting

Deliverable: Working end-to-end solution with README

Sprint 7: Final Demo Preparation (Weeks 13-14)

Goal: Prepare compelling demonstration

Tasks:

- Create demo script and talking points

- Practice presentation
- Final testing and bug fixes

Deliverable: Demo-ready solution and presentation materials

Final Presentation (Week 15)

Each team presents a 15-minute live demo including:

- Problem overview (2 min)
- Solution walkthrough (8 min)
- Live demonstration (5 min)

Datasets

You will work with public data only:

Dataset	Description
Job postings	Public posting dumps and APIs (Kaggle job-posting datasets, Adzuna API, USAJobs API). Title, description, responsibilities, requirements, industry, location
O*NET	US Dept. of Labor occupation database: ~19k task statements, skills, Generalized Work Activities, and occupation–skill links. Group A's reference standard and a source of task–skill priors
ESCO	EU classification of occupations and ~14k skills with hierarchical relations. Group B's reference standard
Lightcast Open Skills	Open, regularly updated skill list with categories; used to check coverage of emerging skills

Not permitted: scraping LinkedIn, Glassdoor, or any site whose terms prohibit it; collecting individual profiles or any personal data. Use only published datasets and official APIs. If unsure whether a source is allowed, ask the instructor before collecting.

Data Contract

Both groups must agree on a shared schema in Sprint 1 so that Sprint 5 can join the taxonomies without rework. The exact design is up to you, but at minimum it should cover:

Entity	Must Capture
Postings	A stable identifier, source, and the raw text
Statements	One row per extracted task or skill statement, linked back to its posting, with a kind flag (task skill)
Topics	Leaf topics with name, description, size, and whatever representation you use to compare topics across groups
Topic membership	Which statements belong to which topics, with an optional strength score

Entity	Must Capture
Hierarchy	Parent–child relations between topics or groups, supporting more than two levels
Task–skill map	Links between task topics and skill topics with a score and the evidence behind it

Evaluation Rubric (100 points)

Category	Points	Criteria
Sprint Deliverables	35	Completing each sprint's working solution
Final Demo	25	Working demonstration, clear presentation
Taxonomy Quality	15	Coherence, coverage, and agreement with reference standards, backed by measured evidence
Code Quality	10	Clean, readable, documented, reproducible code
Claude Code Usage	15	Effective use of AI assistance, good prompting

Sprint Evaluation (35 points total)

Each sprint deliverable is worth approximately 5 points:

- **Complete (5):** Working solution that meets requirements
- **Partial (2-4):** Solution works but missing some features
- **Incomplete (0-1):** Major issues or not submitted

Guidelines for Using Claude Code

Getting Started:

1. Access Claude Code through the shared account (details provided in class)
2. Start with clear, specific requests
3. Review and understand the code Claude generates
4. Iterate—ask for improvements or changes

Best Practices:

- Break big problems into smaller steps
- Describe what you want, not just how to do it
- Ask Claude to explain code you don't understand
- Test generated code before moving on
- Keep a log of useful prompts that worked well

Usage Limits:

- Shared account has 20X usage limit across all teams
- Be mindful of usage—don't waste tokens on unnecessary requests
- If you hit limits, notify the instructor immediately

Guidelines for LLM Use in the Pipeline

Using an LLM inside the pipeline is optional at every step. Students do not have access to Amazon Bedrock; where you do use an LLM, it must be a free-tier or student-credit provider:

- Google AI Studio (Gemini free tier) — recommended default; supports structured output
- GitHub Models via the GitHub Student Developer Pack
- Groq or OpenRouter free tiers for fast, cheap experiments
- Local models via Ollama for unlimited offline runs (slower, lower quality)

Practices:

- Write LLM calls behind a small provider-agnostic interface so the backend can be swapped with one config line
- Always cache LLM responses to disk; never re-pay for the same prompt
- Use the smallest model that passes your quality check; reserve larger models for the steps that need them
- Log token counts per sprint and report them in the sprint report
- Never commit API keys; use environment variables

Important Notes

- All code must be your team's work (with Claude Code assistance)
- Document your Claude Code interactions and LLM prompts in your sprint reports
- Coordinate with the other group early — the data contract is a shared deliverable
- Ask for help early if you're stuck
- Focus on working solutions over perfect solutions

Deliverables Summary

When	What
End of each sprint	Working code + brief report (including LLM token usage)
End of Sprint 4	Complete taxonomy (Group A: tasks; Group B: skills) with evaluation
End of Sprint 5	Joint task–skill map and gold-set evaluation
Week 15	Final demo + all code + documentation

Resources

- Claude Code documentation
- Reference implementation of the framework (provided as an example, not a requirement)
- O*NET database, ESCO downloads, Lightcast Open Skills
- Documentation for embedding, clustering, and LLM libraries of your choice
- Sample prompts and patterns (provided after Sprint 1)
- Office hours: [TBD]